

TS Academy
Backend Developement
Phoenix Cohort Assignment 1
21/03/2026

Assignment: School Library Management API

Tech Stack:

- Node.js
- Express.js
- MongoDB (with Mongoose)

Objective

Build a **RESTful API** for a School Library System that manages:

- Authors
- Books
- Students
- Library Attendants
- Book Borrowing & Returns

Core Entities (Models)

1. Author

Each author can have multiple books.

Fields:

- `name` (String, required)
- `bio` (String)
- `createdAt`

2. Book

Each book:

- Can have **one or more authors**
- Can be **available (in)** or **borrowed (out)**

Fields:

- `title` (String, required)
- `isbn` (String, unique)
- `authors` (Array of ObjectId → Author)
- `status` (Enum: "IN" | "OUT", default: "IN")
- `borrowedBy` (ObjectId → Student, nullable)
- `issuedBy` (ObjectId → Library Attendant, nullable)
- `returnDate` (Date, nullable)
- `createdAt`

3. Student

Library users who can borrow books.

Fields:

- `name` (String, required)
- `email` (String, unique)
- `studentId` (String, unique)
- `createdAt`

4. Library Attendant

Staff responsible for issuing books.

Fields:

- `name` (String, required)
- `staffId` (String, unique)
- `createdAt`

Relationships

- A **Book** → has many **Authors**
- A **Book** → belongs to one **Student** (when borrowed)
- A **Book** → is issued by one **Attendant**

Required Features (Endpoints)

1. Author Endpoints

- `POST /authors` → Create author
- `GET /authors` → Get all authors
- `GET /authors/:id` → Get single author
- `PUT /authors/:id` → Update author
- `DELETE /authors/:id` → Delete author

2. Book Endpoints

- `POST /books` → Create book (must include author IDs)
- `GET /books` → Get all books
- `GET /books/:id` → Get one book (populate authors)
- `PUT /books/:id` → Update book
- `DELETE /books/:id` → Delete book

3. Student Endpoints

- `POST /students` → Create student
- `GET /students` → Get all students
- `GET /students/:id` → Get one student

4. Library Attendant Endpoints

- `POST /attendants` → Create attendant
- `GET /attendants` → Get all attendants

Borrowing & Returning Logic (VERY IMPORTANT)

Borrow Book

Endpoint:

`POST /books/:id/borrow`

Request Body:

```
{
  "studentId": "xxx",
  "attendantId": "xxx",
  "returnDate": "2026-04-01"
}
```

Rules:

- Book must be `"IN"`
- After borrowing:
 - status → `"OUT"`
 - set `borrowedBy`
 - set `issuedBy`

- set `returnDate`

Return Book

Endpoint:

`POST /books/:id/return`

Rules:

- Book must be `"OUT"`
- After return:
 - `status` → `"IN"`
 - clear `borrowedBy`
 - clear `issuedBy`
 - clear `returnDate`

Special Requirement (Important)

When fetching a book:

👉 If book is `"OUT"`, response MUST include:

- Student details
- Library Attendant details
- Return date

👉 Use:

```
.populate("authors")  
.populate("borrowedBy")  
.populate("issuedBy")
```

Project Structure (Expected)

```
/library-system  
/models
```

/controllers
/routes
/services (optional)
/config
server.js

Bonus (For Strong Students)

- Add pagination to `GET /books`
- Add search (by title or author)
- Prevent duplicate ISBN
- Add validation using middleware
- Add overdue check (books past return date)
- Add simple authentication (JWT)

Submission Requirements

- GitHub repository
- README file with:
 - Setup steps
 - API documentation (Postman or Swagger)
- Well-structured code (MVC pattern)

Submit Assignment Here : <https://forms.gle/J3wqYC7rSSuTnfc8>

Date: Saturday 4TH April